

CompEyeAgent

AI 竞品分析 Agent 协作系统

技术架构报告 · 面试准备手册

多 Agent 协同 · DAG 编排 · 可观测性 · 长期记忆

CrewAI · FastAPI · React · ChromaDB · OpenTelemetry · Prometheus

2026 年 6 月

目录

1. 项目概述与业务价值
2. 系统架构全景
3. 技术栈深度解析
4. 核心设计模式
5. 数据流与执行链路
6. 各阶段实现详解
7. 关键技术决策与权衡
8. 可观测性与运维
9. 测试策略
10. 面试高频问题与参考回答
11. 未来优化方向

1. 项目概述与业务价值

1.1 项目定位

CompEyeAgent 是面向产品、市场、战略和研发团队的 AI 竞品分析工作台。系统将公开资料采集、结构化分析、报告撰写和独立质检拆分为多个专职 Agent，通过统一的任务编排和证据模型，完成从需求输入到可信报告交付的完整流程。

核心价值主张：分析过程自动化、结论可追溯（每条结论附来源 URL 和原文片段）、执行过程可观测（每个 Agent 的状态、产物、质检结果可追踪）。

1.2 解决的痛点

- 传统竞品分析依赖人工搜索+手工整理，耗时数天，且容易遗漏关键信息
- 分析结论缺乏来源追溯，难以验证可信度
- 多 Agent 协作过程中缺乏可观测性，无法定位问题环节
- 重复分析同一竞品时无法复用历史知识

1.3 项目规模

代码量：后端 Python ~8000 行 + 前端 TypeScript ~2000 行。测试覆盖：145 个测试用例（135 后端 + 10 前端）。经历 4 个大版本迭代（Phase 1 → Phase 3），27 个子里程碑全部完成。

2. 系统架构全景

2.1 七层架构

系统按职责分为七个层次，每层独立演进：

- 用户入口层：Web App (React) 、CLI、 MCP Server
- 服务层：FastAPI Gateway + SSE 实时事件流
- 任务编排层：Coordinator DAG 调度器 + EventBus 事件总线
- Agent 层：Collector / Analyzer / Writer / Verifier 四个专职 Agent
- 数据层：SQLite 三库分治 + ChromaDB 向量存储
- 可观测层：Prometheus + OTel 分布式追踪
- 平台集成层：MCP Server + 复核队列 + 企业级 Dashboard

2.2 核心执行流程

用户输入 (CompetitorInput)

- RunService.create_run() [SQLite 持久化]
- CoordinatorLoopService.execute() [DAG 状态机]
 - collect 节点: Collector Agent + WebSearchTool
 - analyze 节点: Analyzer Agent
 - write 节点: Writer Agent
 - verify 节点: Verifier Agent (MiMo-V2.5-Pro)
- EventBus → SSE → 前端实时更新
- Provenance Guard + 质检结果 → 报告交付

2.3 三个入口点

- main.py — CLI 入口，直接调用 runner.run_analysis()，适合脚本化批量执行
- app.py — Streamlit UI (Phase 1 遗留)，三列布局，仍可用
- api_app.py — FastAPI 生产入口 (主要)，REST API + SSE + React 前端托管

3. 技术栈深度解析

3.1 后端

- Python 3.11+ (Dockerfile 使用 3.12)
- CrewAI >= 0.60.0 — Agent 编排框架, 负责 Agent 定义、Task 链式执行
- FastAPI >= 0.115.0 — 生产 API 服务器, 支持异步、依赖注入、自动文档
- Pydantic >= 2.0.0 — 数据验证和序列化, 所有模型/schema 均基于 Pydantic
- SQLite3 — 三个独立数据库 (run_store、coordinator_store、source_store)
- ChromaDB >= 0.5.0 — 向量存储, 用于长期记忆的事实嵌入和语义检索
- OpenTelemetry — 分布式追踪和指标采集
- Prometheus Client — 指标暴露

3.2 前端

- React 19 + TypeScript 5.9 + Vite 7
- React Router DOM 7 — SPA 路由
- Vitest 4 — 单元测试
- 纯 CSS (800 行设计系统, CSS 自定义属性)
- EventSource API — SSE 实时订阅

3.3 LLM 模型分工

- Collector / Analyzer / Writer: MiMo-V2.5 — 性价比高, 联网搜索能力强
- Verifier: MiMo-V2.5-Pro — 100 万 Token 上下文 + 深度推理, 用于独立质检
- 设计原则: 让专业的做专业的事, Verifier 不继承 Writer 历史, 防确认偏误

4. 核心设计模式

4.1 DAG 状态机调度

Coordinator 通过 DAG 状态机管理任务执行。每个 DAG 节点有独立的状态 (pending/running/completed/failed/skipped)，通过 `depends_on` 定义依赖关系。调度器循环查找 ready 节点 (pending 且所有依赖 completed)，执行完成后推进状态。失败节点的后代自动标记为 skipped。

关键实现：coordinator_loop.py 的 `_execute_dag()` 方法是一个 `while True` 循环，每轮调用 `_ready_nodes()` 找到可执行节点，通过 `_execute_node_with_retry()` 执行，支持 `max_retries` 配置。

4.2 Scratchpad 数据传递

节点间通过 Scratchpad (SQLite-backed) 传递数据，而非 CrewAI 的 context 链。每个节点读取上游的 Scratchpad 产物，写入自己的输出。路径约定：collect/raw.json、analyze/findings.json、write/report.md、verify/verifier.json。这解耦了节点间的依赖，支持独立重试单个节点。

4.3 Provenance Guard (溯源守卫)

runner.py 中的多层校验机制，确保报告结论可追溯：

1. Regex 检查：报告必须包含来源索引区块、至少一个 URL、来源标签数匹配结论数
2. Verifier JSON 解析：检查 passed/confidence/issues
3. 失败时自动重跑 Writer+Verifier 一次（注入具体问题描述）

4.4 EventBus 双写模式

事件通过 `_emit()` 方法双写：先 SQLite 持久化（支持断线重连），再 EventBus 内存队列推送（毫秒级延迟）。SSE 端点优先 await 事件队列，队列不存在时自动回退到 SQLite 轮询。通过 `loop.call_soon_threadsafe()` 桥接同步 Coordinator 线程和 async SSE 端点。

4.5 CircuitBreaker 熔断器

每个 LLM 提供者一个 CircuitBreaker 实例，三态机：

- closed（正常）：调用通过，失败计数
- open（熔断）：连续失败达阈值后，拒绝调用，进入冷却期
- half_open（探测）：冷却期后允许一次调用，成功则关闭，失败则重新打开

集成到 ModelRegistry 的 `create_llm()` 中，主模型熔断时自动 fallback 到次选模型。

4.6 Protocol 存储抽象

storage/protocols.py 定义三个 `@runtime_checkable` Protocol：RunStoreProtocol、CoordinatorStoreProtocol、SourceStoreProtocol。所有消费端 (RunService、CoordinatorLoopService 等) 依赖 Protocol 而非具体 SQLite 类。PostgreSQL 迁移只需新增实现类并替换实例化，零代码改动。

5. 数据流与执行链路

5.1 完整数据流

用户输入 → CompetitorInput (Pydantic)

- RunService.create_run() → SQLite run_store + DAG 初始化
- background_tasks → RunService.execute_run()
- MemoryService.query_for_run() → ChromaDB 语义检索历史事实
- CoordinatorLoopService.execute()
- _execute_dag() 状态机循环
 - collect 节点: Collector Agent + WebSearchTool + Evidence Index
 - 输出写入 Scratchpad: collect/raw.json
 - analyze 节点: 读 collect/raw.json, 分析
 - 输出写入 Scratchpad: analyze/findings.json
 - write 节点: 读 analyze/findings.json, 生成报告
 - 输出写入 Scratchpad: write/report.md
 - verify 节点: 读 write/report.md, 质检
 - 输出写入 Scratchpad: verify/verifier.json
- _persist_success() → 持久化产物 + 来源索引
- MemoryService.ingest_completed_run() → ChromaDB 存储已验证事实
- EventBus.publish() → SSE 推送 → 前端实时更新

5.2 三库分治

- run_store.sqlite3 — analysis_runs、agent_events、artifacts、source_references、review_queue
- coordinator_store.sqlite3 — dag_nodes、scratchpad_items
- source_store.sqlite3 — source_seeds、raw_documents、evidence_items、source_fetch_events

每个库使用 threading.RLock 保证线程安全，auto-create 表结构。

6. 各阶段实现详解

Phase 1: 可运行 MVP

验证核心闭环：四类 Agent 协作完成竞品分析。

- CrewAI sequential chain: Collector → Analyzer → Writer → Verifier
- 每个 Task 通过 context=[previous_task] 链式传递数据
- runner.py 实现 Provenance Guard + 最小重写闭环
- CLI (main.py) + Streamlit UI (app.py)

Phase 1.5: 产品 Demo

将 MVP 包装为在线产品形态。

- FastAPI API 网关 + React Web App (Demo/Dashboard/Report 三页面)
- SSE 事件流 + SQLite run store + 前后端同服务托管
- Docker 多阶段构建 + Render.com 云部署

Phase 2: 任务编排增强

增强后端任务编排和可观测数据源。

- Source Layer: 5 种 Connector (Jina/NewsAPI/GitHub/RSS/Reddit) + 证据提取
- DAG 调度器: 4 节点线性链 + 依赖推进 + 节点级重试
- 逐节点执行器: 每个节点独立 CrewAI Crew, 通过 Scratchpad 传递上下文
- EventBus: asyncio.Queue 内存事件队列, SSE 零轮询推送
- OTel: Prometheus /metrics + 分布式追踪 (run/node span)

Phase 3: 企业级平台

提升稳定性、治理能力和平台集成。

- 存储抽象层: Protocol 接口, SQLite→PG 可切换
- 韧性设计: CircuitBreaker + 节点超时 + 部分结果交付
- 多模型降级: ModelRegistry 按优先级 fallback + 熔断器健康追踪
- 完整 OTel: LLM 调用 span + per-model token/延迟指标
- 人工复核队列: review_queue 表 + 审核 API
- 长期记忆: ChromaDB 向量存储 + 事实提取 + 跨 run 语义检索
- 企业级 Dashboard: 概览/复核/成本 3 个新页面
- MCP Server: FastMCP 服务器, 9 个工具, stdio + HTTP/SSE 传输

7. 关键技术决策与权衡

7.1 为什么用 CrewAI 而非自研 Agent 框架？

Phase 1 选择 CrewAI 是为了快速验证核心闭环。CrewAI 提供了 Agent 定义、Task 链式执行、context 传递等开箱即用的能力。Phase 2 通过 DAG 调度器 + 逐节点执行器逐步解耦了对 CrewAI 的依赖，每个节点的 Crew 是独立创建的，不再是模块级单例。未来可替换为自研 Agent 运行时。

7.2 为什么用 SQLite 而非 PostgreSQL？

SQLite 零运维、单文件部署，适合 MVP 和中小规模。三个独立数据库避免了不同关注域的锁竞争。Phase 3 通过 Protocol 抽象层为 PostgreSQL 迁移做好了准备，消费端代码无需改动。

7.3 为什么 EventBus 用 asyncio.Queue 而非 Redis Pub/Sub？

单进程部署下 asyncio.Queue 足够，延迟更低、无外部依赖。双写模式 (SQLite + Queue) 保证了持久化和实时性的平衡。多进程部署时可替换为 Redis Pub/Sub，接口不变。

7.4 为什么 Verifier 用不同模型？

MiMo-V2.5-Pro	拥有	100	万	Token
---------------	----	-----	---	-------

上下文和更强的推理能力，适合需要深度逻辑校验的质检环节。独立模型实例不继承历史，强制独立判断，防确认偏误。这是参考了 Claude Code 的 Verification Agent 模式。

				Writer
--	--	--	--	--------

7.5 部分结果交付的设计考量

当	verify	节点失败但	write
---	--------	-------	-------

已完成时，系统返回草稿报告 (needs_review) 而非完全失败。这保证了即使质检不通过，用户仍能看到分析结果。配合人工复核队列，用户可以决定是否接受草稿或触发重写。

8. 可观测性与运维

8.1 Prometheus 指标

- `compeye_runs_total{status}` — 总 run 数
- `compeye_run_duration_seconds{status}` — run 执行时长
- `compeye_node_duration_seconds{node_key}` — 节点执行时长
- `compeye_node_retries_total{node_key}` — 节点重试次数
- `compeye_events_total{event_type}` — 事件总数
- `compeye_active_runs` — 当前活跃 run 数
- `compeye_llm_calls_total{model, node_key, status}` — LLM 调用次数
- `compeye_llm_call_duration_seconds{model, node_key}` — LLM 调用延迟
- `compeye_llm_tokens_total{model, node_key, direction}` — Token 用量

8.2 OTEL 分布式追踪

- `run.execute span` — 完整 run 执行链路
- `node.{collect|analyze|write|verify} span` — 单节点执行
- `llm.call span` — 单次 LLM API 调用 (`model`、`prompt_length`、`duration`)

默认关闭，通过 `COMPETEYE_OTEL_ENABLED=true` 启用，支持 OTLP export。

8.3 SSE 事件流

11

种事件类型覆盖完整生命周期：`run.created`、`run.started`、`agent.started`、`agent.progress`、`agent.completed`、`agent retrying`、`verifier.issue`、`artifact.ready`、`run.completed`、`run.failed`、`run.cancelled`。前端通过 EventSource API 订阅，DashboardPage 实时更新。

9. 测试策略

9.1 后端测试 (135 个)

- test_coordinator_foundation.py — DAG 创建、Scratchpad CRUD、API 端点、节点调度
- test_verification.py — Provenance Guard、Verifier 解析
- test_resilience.py — CircuitBreaker 状态机、超时
- test_model_registry.py — 模型注册表、fallback 链
- test_telemetry.py — Prometheus 指标、OTel span
- test_event_bus.py — 内存事件队列
- test_store_protocols.py — Protocol 一致性验证
- test_memory_service.py — ChromaDB 向量存储、事实提取
- test_mcp_server.py — MCP 工具函数
- test_source_layer.py / test_source_refresh.py / test_source_seed_registry.py — 来源层
- test_static_frontend.py — FastAPI 静态文件托管

9.2 前端测试 (3 个)

- runData.test.ts — buildCreateRunRequest、deriveStageStates、selectArtifacts

9.3 测试策略

- 每个 service 层模块独立可测，mock 外部依赖 (CrewAI、LLM API)
- SQLite 使用 tempfile.TemporaryDirectory 隔离
- ChromaDB 使用 in_memory=True 模式避免模型下载
- MCP 测试使用 @patch 隔离全局状态

10. 面试高频问题与参考回答

架构设计类

Q: 介绍一下这个项目的整体架构？

A: 系统采用七层架构：用户入口 (Web/CLI/MCP)、服务层 (FastAPI)、任务编排层 (DAG 调度器 + EventBus)、Agent 层 (Collector/Analyzer/Writer/Verifier)、数据层 (SQLite 三库 + ChromaDB)、可观测层 (Prometheus + OTel)、平台集成层 (MCP Server + Dashboard)。核心执行流程是：用户输入 → DAG 调度器逐节点执行 Agent → Scratchpad 传递数据 → Provenance Guard 校验 → 报告交付。

Q: 为什么选择 DAG 而不是简单的线性流水线？

A: Phase 1 确实是线性流水线 (CrewAI sequential)，但 Phase 2 升级为 DAG 有两个原因：1) DAG 支持节点级重试，单个节点失败可以独立重跑而不影响其他节点；2) DAG 天然支持未来扩展为并行执行 (如同时采集多个竞品的信息)。当前是 4 节点线性链 collect→analyze→write→verify，但架构已支持任意拓扑。

Q: Scratchpad 是什么？为什么不用 CrewAI 的 context 传递？

A: Scratchpad 是 SQLite-backed 的键值存储，路径约定为 node_key/filename.ext。不用 CrewAI context 的原因：1) context 是内存中的 Python 对象引用，节点间耦合强；2) 无法独立重试单个节点 (需要重建整条 context 链)；3) Scratchpad 持久化后支持断点续跑和 Run Inspector 查询。

技术实现类

Q: EventBus 是怎么实现的？为什么要双写？

A: EventBus 基于 asyncio.Queue，通过 loop.call_soon_threadsafe() 桥接同步线程和 async SSE。双写 (SQLite + Queue) 的原因：SQLite 持久化支持断线重连 (after_event_id 游标)，Queue 提供毫秒级推送。SSE 端点优先 await Queue，Queue 不存在时回退到 SQLite 轮询。多进程部署时可替换为 Redis Pub/Sub。

Q: CircuitBreaker 的实现细节？

A: 三态机：closed (正常计数) → open (拒绝调用，进入冷却) → half_open (允许一次探测)。阈值和冷却期可配置。集成到 ModelRegistry 中，主模型熔断时自动 fallback。线程安全，使用 threading.Lock 保护状态转换。

Q: Provenance Guard 具体检查什么？

A: 三层校验：1) Regex 检查报告必须有来源索引区块和至少一个 URL；2) 来源标签数必须匹配结论数；3) Verifier JSON 解析 passed/confidence/issues。失败时自动重跑 Writer+Verifier 一次，注入具体问题描述。

数据与存储类

Q: 为什么用三个 SQLite 数据库？

A: 关注域分离：run_store 管运行记录和事件，coordinator_store 管 DAG 和 Scratchpad，source_store 管来源情报。避免不同写入模式的锁竞争。每个库使用 threading.RLock 保证线程安全。

Q: 长期记忆是怎么实现的?

A: ChromaDB 向量存储 + 事实提取。Run 完成后自动提取报告中的 claim-like 行, 过滤 confidence >= 70 的已验证事实, 嵌入存储。新 run 开始时语义检索相关历史事实, 注入 Collector/Analyzer 的提示词。使用 SimpleEmbedding 作为 fallback (生产环境可换 sentence-transformers)。

Q: 多模型降级是怎么实现的?

A: ModelRegistry 按 agent role 注册多个 provider, 每个有优先级。create_llm() 按优先级尝试, 通过 CircuitBreaker 检查健康状态, 熔断的 provider 自动跳过。支持环境变量和 YAML 两种配置方式。

工程实践类

Q: 怎么保证代码质量?

A: 145 个自动化测试覆盖所有核心模块。Protocol 接口确保存储层可替换。每个 Phase 都有 E2E 验证记录。AGENTS.md 定义了 Karpathy 编码规范。CI/CD 通过 GitHub Actions 运行 pytest。

Q: 遇到过什么技术挑战? 怎么解决的?

A: 1) CrewAI task context 耦合 → 升级为 Scratchpad 数据传递; 2) 同步 CrewAI kickoff 与 async SSE → EventBus + call_soon_threadsafe 桥接; 3) Windows SQLite 文件锁定 → tempfile.TemporaryDirectory + PermissionError 容错; 4) ChromaDB 首次下载 ONNX 模型太慢 → SimpleEmbedding fallback。

Q: 如果要支持 100 并发用户, 需要改什么?

A: 1) SQLite → PostgreSQL (Protocol 接口已就绪, 只需新增实现类); 2) asyncio.Queue → Redis Pub/Sub (接口兼容); 3) BackgroundTasks → Celery/RQ 任务队列; 4) 添加连接池 (asyncpg) 和缓存层 (Redis)。存储抽象层的设计使得这些改动互不影响。

MCP 和 Agent 生态类

Q: MCP Server 是什么? 你实现了哪些工具?

A: MCP (Model Context Protocol) 是 Anthropic 定义的 Agent 互操作协议。我用 FastMCP 实现了 9 个工具: create_run、get_run、get_report、get_verification、list_runs、get_sources、get_scratchpad、cancel_run。支持 stdio (Claude Desktop 直接调用) 和 HTTP/SSE (远程访问) 两种传输。

11. 未来优化方向

- PostgreSQL 迁移 — Protocol 接口已就绪，需新增 PgRunStore + Alembic 迁移
- RBAC 权限系统 — 用户模型 + JWT 认证 + 权限中间件
- 飞书/钉钉接入 — 协作平台机器人
- Webhook 通知 — run 完成/复核时推送外部通知
- Grafana Dashboard — OTel 数据可视化
- Semantic Diff — 对比两次 run 的报告差异